

RAGScope

A reference manual for the public RAG benchmarking harness

This document is the single source of truth for understanding the RAGScope codebase. It walks through every layer of the system - from why the project exists, through the architecture and each individual file's role, to a free-tier troubleshooting playbook - in the order a new contributor would need to read it.

Author: ImtiazX

Generated: 2026-06-05

License: MIT

Repository: github.com/ImtiazX/ragscope

Live app: ragscope.vercel.app

Table of Contents

1. Introduction	What the project is and is not
2. Background	Why this project exists, problem statement
3. Core functional features	What the user can do, end to end
4. Use cases	Concrete scenarios the harness is built for
5. Technical architecture	High-level diagram and request flow
6. Components and modules	Backend and frontend module map
7. Codebase walkthrough	File-by-file role and responsibility
8. Library reference	Every dependency and why it is needed
9. How the files connect	Import graph and runtime call chains
10. Troubleshooting playbook	Free-tier failure modes by service
11. Operational runbook	Deploys, rotations, common chores
12. Interview prep capsule	Talking points and the design decisions

1. Introduction

1.1 What RAGScope is

RAGScope is a public, web-hosted benchmarking harness for Retrieval-Augmented Generation (RAG) systems. A user uploads a document corpus (PDF or plain text), asks a question, and the harness runs the same question through up to four different retrieval strategies in parallel - then scores each result with RAGAS using *gpt-4o-mini* as the judge. The user sees exactly which strategy wins on faithfulness, context utilization, answer relevancy, and end-to-end latency for *their* data.

1.2 What RAGScope is not

RAGScope is not a RAG product, not a chatbot framework, not an agent system, and not a vector-database wrapper. Everything it does serves one purpose: produce trustworthy comparative measurements between retrieval strategies on a real corpus. There is no agent loop, no tool use, no streaming dialogue. The live-chat surface in Step 4 exists only so the winning strategy translates into a felt experience.

1.3 Audience for this manual

Written for two readers:

- **A new contributor** who has just cloned the repository and needs to understand the codebase before opening their first PR.
- **An interviewee** (the original author of the project) who needs to confidently answer architecture, design-decision, and trade-off questions about every component.

Every section therefore explains *both* what something does and *why* it was built that way.

1.4 How to read this document

- Sections 1-4 are conceptual. Read once.
- Sections 5-9 are architectural. Skim once, then refer back as you navigate the source.
- Section 10 is a troubleshooting playbook. Read once before a deploy, and again whenever a free-tier service misbehaves.
- Section 12 is an interview-prep capsule that distils the rest into talking points.

2. Background

2.1 The problem RAGScope solves

RAG is now the default pattern for grounding LLM answers in external knowledge. The decision every team faces is: *which retrieval strategy works best on our corpus?* Naive cosine similarity, HyDE, multi-query, hybrid BM25 + dense, contextual compression - every blog post claims its favourite is the best, but the right answer depends on the documents and questions in front of you. Picking blind is expensive: the wrong choice means either hallucinations in production or paying for unnecessary LLM calls during retrieval.

RAGScope makes the decision empirical. Upload the actual documents, ask the actual questions, and read the actual scores. The harness is the measuring tape that the rest of the RAG ecosystem treats as a given.

2.2 Design tenets

- **Measurement first.** Every architectural decision is judged against whether it preserves the integrity of the measurement.
- **Deterministic pipeline.** No agents, no tool use, no LLM loops. The eval path is a fixed sequence of steps so two runs on the same input produce comparable scores.
- **Auto-discoverable strategies.** A retriever, chunker, ingestor, or LLM provider plugs in by extending a base class and using a single decorator. No hardcoded strategy lists exist anywhere - the API and UI populate dynamically.
- **Honest free tier.** Twelve strategy runs and five chat questions per day per browser fingerprint, no signup, no email. Heavier use is unlocked by pasting your own key (which stays in the browser).
- **Public source of truth.** The repository, the live app, the API documentation, and this manual are all public. Nothing is hidden behind a CONFIDENTIAL.md.

2.3 Why these four retrieval strategies

The four selected strategies are not exhaustive - they are the four most-cited *different shapes* of retrieval in the RAG literature. Picking one of each shape, instead of three variations on cosine similarity, means the benchmark answers a useful question for the user: *which style of retrieval fits my data?*

Shape	Strategy	What kind of question it answers
Baseline dense	Naive RAG	How does pure cosine similarity do on this corpus?
Query rewriting	HyDE	Do my questions and my documents share vocabulary?
Query expansion	Multi-query	Does a single phrasing miss relevant passages?
Sparse + dense fusion	Hybrid BM25 + dense	Do exact keywords or names matter beyond semantics?

2.4 Why contextual compression is a separate axis

Compression is not a fifth retrieval strategy. It is a post-retrieval LLM filter that removes irrelevant sentences from each chunk before they reach the answerer. Architecturally it sits *after* any of the four retrievers and is combinable with all of them. Surfacing it as a separate toggle (rather than inflating the strategy count to five) keeps the experiment design clean: a user can compare HyDE vs. HyDE + compression on equal footing, which would be invisible if it were a different row in the strategy list.

3. Core functional features

3.1 Step 1 - Upload

- Drag-and-drop or file-picker upload of one or more PDF / TXT files (10 MB combined limit, enforced at FastAPI request and application level).
- Per-corpus content hash (SHA-256) so a repeat upload of the same files is detected as a duplicate and short-circuits with HTTP 200 instead of re-running the pipeline.
- Configurable chunker (fixed-size, semantic, or hierarchical) with tunable parameters rendered dynamically from the backend `param_schema`.

3.2 Step 2 - Configure benchmark

- Select one or more of the four retrieval strategies. Selecting N strategies costs N runs against the daily quota.
- Tune retrieval parameters (`top_k`, `num_variants` for multi-query, `bm25_weight` for hybrid, `hypothetical_doc_length` for HyDE, etc.) via the dynamic parameter form.
- Toggle contextual compression independently of the strategy choice. Enabling it does not consume an additional run.
- Enter the benchmark question and submit. The browser receives N `run_ids` immediately and starts polling.

3.3 Step 3 - Results dashboard

- Live, streaming dashboard: results appear strategy by strategy as each background task completes, instead of waiting for the slowest one.
- Radar chart with three axes (faithfulness, context utilization, answer relevancy) overlaid for every completed run.
- Latency bar chart sorted fastest to slowest, with plain-English interpretation per bar.
- Sortable comparison table, color-coded best/worst per metric, click-to-select per row.
- Score cards with count-up animation and plain-English interpretation under each metric.
- Animated 'Winner' badge for the highest weighted-average run, with a one-line explanation of why it won.
- Session-scoped history persisted in `localStorage` so the comparison view accumulates across submissions until the user clears it.

3.4 Step 4 - Live chat

- Conversational interface against the same corpus, defaulting to the winning strategy.
- Collapsible config panel to switch strategy or change parameters mid-conversation.
- Tier 1 counter (5 questions / day) shown next to the input. The backend is authoritative; the frontend counter is for display.

- Per-message metadata: strategy used, number of chunks retrieved, expanding to show the chunks themselves.

3.5 Global features

- Dark / light / system theme with no-flash injection script in the root layout.tsx.
- Optional ambient audio (off by default, persisted in localStorage).
- Soft click sound effects on interactive elements.
- BYOK settings drawer slide-in from the right.
- Toast notification system for async events (upload success, rate-limit hit, network error).
- Skeleton loading states on all API calls.
- Plain-English tooltips on every parameter control.
- Tier 0 dev bypass via ?dev=<token> URL param.

4. Use cases

4.1 Choosing a retrieval strategy for a new RAG project

A team starting a RAG project loads a representative sample of the production corpus (say 20-50 pages) and runs five or six questions an end user might ask. The benchmark reveals whether HyDE provides any lift on this corpus, whether BM25 picks up identifiers that dense similarity smooths over, and whether compression is worth the extra LLM cost. The decision is data-driven instead of folklore-driven.

4.2 Auditing an existing pipeline

A team already in production suspects retrieval is the bottleneck (low faithfulness scores, frequent hallucinations). They paste the same questions the LLM has been seeing in production and compare the production strategy (likely naive) against HyDE and hybrid on the production corpus. If faithfulness jumps from 0.6 to 0.9 with hybrid, the experiment is the business case for swapping retrievers.

4.3 Teaching RAG fundamentals

Instructors and bootcamps point students at RAGScope to see the *shape* of each strategy without building it. The docs page explains the math behind each metric (with LaTeX formulae). Students upload a paper, run the benchmark, see the difference between BM25 and cosine in the chart, and internalise the trade-off before writing a line of code.

4.4 Interview portfolio piece

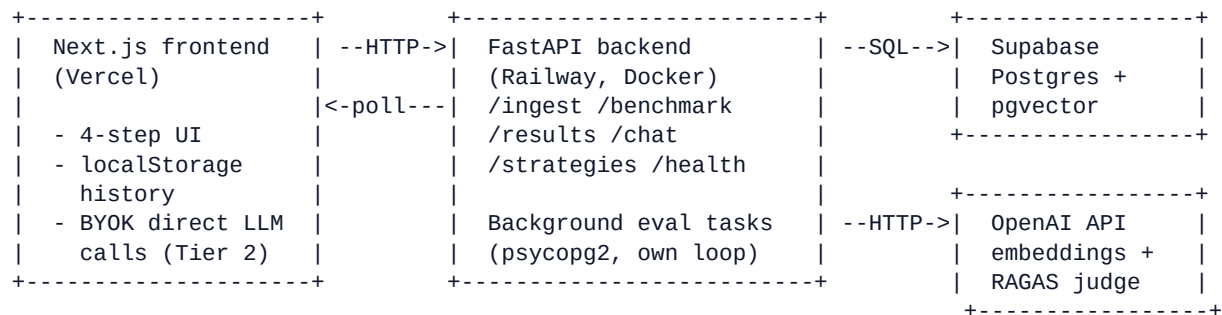
The original author uses RAGScope as a portfolio project that demonstrates: production-grade FastAPI with async background tasks, vector search with pgvector, RAGAS evaluation pipelines, Next.js 14 App Router, polished UX with dynamic forms, secure BYOK that never touches the server, sane rate limiting with fingerprint hashing, three-platform deployment (Railway, Vercel, Supabase), and an extensible plugin architecture that earns its keep when new retrievers are added.

4.5 Reproducible research baseline

Researchers comparing new retrieval techniques against existing ones need a baseline. RAGScope ships the four most-cited strategies behind one unified eval pipeline so the only variable is the new technique. The auto-discovery pattern means a new strategy plugs in with one decorator - no surgery on the rest of the codebase.

5. Technical architecture

5.1 System diagram



5.2 Three layers, three runtimes

- **Frontend (Next.js on Vercel).** Renders the UI, owns browser-only state (theme, history, BYOK key), polls the backend for results, and on BYOK calls the LLM provider *directly* from the browser so the user's key never transits the backend.
- **Backend (FastAPI on Railway).** Stateless request handler. Talks to Postgres + pgvector for storage and to the OpenAI API for embeddings, retrieval LLM, and RAGAS judging. Schedules background tasks for the long-running RAGAS eval.
- **Database (Supabase Postgres + pgvector).** Single source of truth for the corpus, the benchmark runs, and the rate-limit counters. The backend creates all tables on first startup via `create_tables()`; no manual migrations.

5.3 The benchmark request flow

- **POST /ingest.** Multipart upload. Backend hashes the bytes (SHA-256), checks if that corpus already exists, otherwise extracts text via the ingestor registry, chunks via the chunker registry, embeds every chunk concurrently via `asyncio.gather`, and bulk-inserts into `corpus_chunks` via `asyncpg executemany`.
- **POST /benchmark.** Validates the corpus, validates every strategy in the request, checks the daily-run quota against the fingerprint hash, then opens one `benchmark_runs` row and schedules one background task per strategy. Returns `{run_ids: [...]}` with HTTP 202.
- **Background task.** Runs on a dedicated event loop in a worker thread (see Section 5.4). Loads the corpus, runs retrieval and optional compression, generates an answer with `gpt-4o-mini`, scores three RAGAS metrics one at a time, and writes the result row with `status='completed'`.
- **GET /results/{run_id}.** Polled by the frontend every ~1s. Returns the full row including status. Frontend stops polling when status is 'completed' or 'failed'.
- **POST /chat.** Same retrieval / compression / answer generation but without scoring. Used by Step 4 for the live conversational surface. Counts against the chat quota (5/day guest), not the run quota.

5.4 The background-task event loop pattern

The single most subtle piece of the backend. RAGAS evaluation is long-running (sometimes 30-90 seconds) so it must not block the request thread. FastAPI's BackgroundTasks dispatches non-async callables on a worker thread via anyio. That worker thread has no asyncio event loop. The sync wrapper `run_evaluation` creates a fresh event loop with `asyncio.DefaultEventLoopPolicy().new_event_loop()`, sets it as the current loop for the thread, wraps the async implementation in a Task via `loop.create_task()`, and drives it with `loop.run_until_complete(task)`.

Why the Task wrapper matters. *asyncpg's connect / release code calls `asyncio.timeout()` internally. On Python 3.11+, `asyncio.timeout()` calls `asyncio.current_task()` and raises `RuntimeError("Timeout should be used inside a task")` if it returns `None`. `loop.run_until_complete(bare_coroutine)` drives the coroutine without a Task wrapper, so `current_task()` is `None`. Creating a Task first solves the problem.*

The async implementation uses synchronous **psycopg2** for all DB calls inside the task - not `asyncpg`. Why: on the previous Render host (Python 3.14), `asyncpg's connect` path called `asyncio.timeout()` in a way that raised *even with the Task wrapper*, under any non-trivial concurrent load. `psycopg2` is fully synchronous and never touches `asyncio`, so it cannot trip the failure mode by construction. The brief blocking of the dedicated task loop is acceptable because no other coroutine is competing for it. The current Railway deployment pins Python 3.11.9 (Dockerfile), so the original bug is dormant; `psycopg2` stays as a safety net.

5.5 RAGAS per-metric isolation

RAGAS 0.1.21 evaluates all metrics in a single `evaluate()` call by default. Its internal worker coroutines also call `asyncio.timeout()`. If `current_task()` returns `None` at the wrong moment in a worker, the whole call raises and the entire benchmark run dies, even though two of three metrics succeeded. `_run_ragas` in `backend/eval/ragas_runner.py` fixes this by running each of the three metrics in its own `evaluate(dataset, metrics=[one_metric])` call, each wrapped in `try / except BaseException`. A failure in one metric records `float('nan')` for that metric and proceeds. The result row carries NaN for the broken metric and real numbers for the rest.

5.6 Rate limiting and identity

Guest tier identity is a SHA-256 hash of `(client_ip + ':' + x_fingerprint_header)`. Browser fingerprint alone is defeated by clearing site data; IP alone is defeated by switching networks; the composite hash requires the evader to do both at once. The hash is irreversible so no plaintext PII is stored. Counters live in `rate_limit_counters` with composite primary key `(fingerprint_hash, date)` so the limit resets at midnight UTC and old rows are leftover for analytics.

6. Components and modules

6.1 Backend package map

backend/main.py	FastAPI app factory, CORS, lifespan, /health and /strategies. Intentionally thin - only wiring and configuration.
backend/core/	Config (env vars), auth (dev token), rate_limiter (fingerprint hashing and daily counters), database (asyncpg pool + schema).
backend/ingest/	File loaders behind a registry. One module per file format (pdf, txt). Each extends BaseIngestor and uses @register.
backend/chunkers/	Text splitters behind a registry. Three strategies: fixed_size, semantic (embedding-similarity boundaries), hierarchical (parent/child two-level).
backend/retrieval/	Retrieval strategies behind a registry. Four strategies (naive, hyde, multiquery, hybrid) plus contextual_compression which is a post-retrieval processor outside the registry.
backend/llm/	LLM provider abstraction. OpenAI provider used by the backend; Anthropic provider available for BYOK. Each exposes async + sync variants of complete() and embed().
backend/eval/	ragas_runner - the background-task implementation. The most subtle file in the codebase. See Section 5.4.
backend/routers/	FastAPI routers, one per domain: ingest, benchmark, results, chat. Each owns its own request validation and response shape.

6.2 Frontend package map

frontend/app/layout.tsx	Root layout. No-flash theme inject, font load, ThemeProvider + AudioManagerProvider + UIContextProvider, mounts BYOKDrawer and ToastDisplay at root for global access.
frontend/app/page.tsx	Landing page. Hero, Why, Features, CTA, Footer. Renders SnowflakeBackground particles and the TierSelectionModal.
frontend/app/app/page.tsx	App shell. Wraps the four steps in AppContextProvider. Manages step navigation and shows the first-visit tier modal.
frontend/app/app/steps/	Step1Upload, Step2Configure, Step3Results, Step4Chat. Each step reads and writes shared state via useAppContext().

frontend/app/docs/page.tsx	Public docs page. How it works (three phases), metric explanations with KaTeX formulae, retrieval strategy descriptions, FAQ.
frontend/components/	Nav, StepIndicator, ParamForm (dynamic form from param_schema), BYOKDrawer, TierModal, TierSelectionModal, ErrorBoundary, ThemeProvider, AudioManager, ToastDisplay, and decorative backgrounds (Snowflake, RAGFlow, Particle).
frontend/context/	AppContext (corpus, runs, history, byok), UIContext (toasts, drawer state, theme accessor).
frontend/lib/api.ts	Typed fetch client for every backend endpoint. Attaches X-Dev-Token and X-Fingerprint headers automatically.
frontend/lib/utils.ts	Misc helpers (className join, format utilities).

7. Codebase walkthrough

Every non-trivial source file in the repository, in import order. Generated assets (.next/, node_modules/, .pytest_cache/) are omitted.

7.1 Backend - bootstrapping and config

backend/main.py	FastAPI app entry. Wires CORS, lifespan (create_tables on startup, close_pool on shutdown), all routers, and the /health and /strategies utility endpoints.
backend/core/config.py	The only file that reads environment variables. Defines the Settings pydantic-settings class and the module-level settings singleton imported everywhere else.
backend/core/auth.py	Dev token validation. SHA-256 hash on both sides; hmac.compare_digest for timing safety. Exposes the get_dev_access FastAPI dependency.
backend/core/rate_limiter.py	Daily run and chat limit constants. get_fingerprint_hash dependency turns (IP + X-Fingerprint) into a SHA-256 identity.
backend/core/database.py	asyncpg pool singleton (get_pool), psycopg2 sync connection helper (make_sync_connection), DDL via create_tables, and CRUD helpers for the three tables (corpus_chunks, benchmark_runs, rate_limit_counters).

7.2 Backend - ingest pipeline

backend/ingest/base.py	BaseIngestor abstract class and the @register decorator. Module-level registry dict populated at import.
backend/ingest/pdf.py	PdfIngestor. Wraps bytes in io.BytesIO, uses pypdf.PdfReader, returns one string per non-blank page.
backend/ingest/txt.py	TxtIngestor. UTF-8 decode, returns single-element list to match the multi-page interface of PdfIngestor.
backend/ingest/registry.py	Side-effect import module. Imports pdf and txt so their @register decorators run.

7.3 Backend - chunkers

backend/chunkers/base.py	BaseChunker abstract class, RetrievalResult, @register. Enforces param_schema at registration time.
---------------------------------	---

backend/chunkers/fixed_size.py	FixedSizeChunker. Whitespace tokenization, fixed-size windows with overlap. Baseline chunker.
backend/chunkers/semantic.py	SemanticChunker. Embeds each sentence, splits where adjacent similarity drops below threshold. Calls the embeddings API.
backend/chunkers/hierarchical.py	HierarchicalChunker. Two levels: parent (1024 tokens default) for context, child (256 tokens default) for retrieval precision.
backend/chunkers/registry.py	Side-effect import module.

7.4 Backend - retrieval strategies

backend/retrieval/base.py	BaseRetriever abstract class, RetrievalResult dataclass, @register decorator. Same registry pattern as ingestors and chunkers.
backend/retrieval/naive.py	NaiveRetriever. Embeds query, cosine similarity vs every stored chunk, returns top-k. Baseline.
backend/retrieval/hyde.py	HydeRetriever. LLM call to generate hypothetical answer, embed that, retrieve against it. hypothetical_doc_length parameter (short / medium / long).
backend/retrieval/multiquery.py	MultiQueryRetriever. One LLM call to generate num_variants rewordings, parallel embed via gather, merge by best score.
backend/retrieval/hybrid.py	HybridRetriever. BM25 (rank-bm25) and dense run in parallel, fused with Reciprocal Rank Fusion. bm25_weight and rrf_k parameters.
backend/retrieval/contextual_compression.py	ContextualCompressor. NOT a BaseRetriever. Post-retrieval filter: per-chunk LLM call extracts only query-relevant sentences. Dropped chunks fall below min_relevance_length.
backend/retrieval/registry.py	Side-effect import module.

7.5 Backend - LLM providers

backend/llm/base.py	BaseLLMProvider abstract class and @register decorator.
----------------------------	---

backend/llm/openai_provider.py	OpenAIProvider. async + sync variants of complete() and embed(). Uses httpx.AsyncClient or httpx.Client per call. The sync variants are required by the background task path.
backend/llm/anthropic_provider.py	AnthropicProvider. Async + sync complete() only - Anthropic has no public embeddings API, so embed() raises NotImplementedError. Used by Tier 2 BYOK.
backend/llm/registry.py	Side-effect import module.

7.6 Backend - evaluation

backend/eval/ragas_runner.py	The background task. run_evaluation is the sync entry point registered with BackgroundTasks; _run_evaluation_async is the async implementation. _run_ragas runs each metric in isolation. get_run is the helper used by the results router.
-------------------------------------	---

7.7 Backend - routers

backend/routers/ingest.py	POST /ingest. Read all files, enforce 10MB cap, hash, deduplicate, ingest via registry, chunk, embed via asyncio.gather, bulk-insert via asyncpg executemany.
backend/routers/benchmark.py	POST /benchmark. Validate corpus + strategies, check guest quota for N strategies, open N rows, schedule N background tasks, increment counter by N, return all run_ids with 202.
backend/routers/results.py	GET /results/{run_id}. Reads the full benchmark_runs row. Frontend polls this endpoint until status is terminal.
backend/routers/chat.py	POST /chat. Same retrieval + compression + answer generation as the benchmark, but no RAGAS scoring and no benchmark_runs row. Counts against the 5/day chat quota.

7.8 Frontend - shell

frontend/app/layout.tsx	Root layout. No-flash theme inject script, Inter font, Provider stack (Theme, Audio, UI), mounts BYOKDrawer + Toast.
frontend/app/page.tsx	Landing page (/) with Snowflake particles and tier modal.

frontend/app/app/page.tsx	App shell (/app). Wraps the four steps in AppContextProvider, manages forward/backward step navigation, shows first-visit TierModal.
frontend/app/docs/page.tsx	Public docs (/docs). Explains how it works, metric formulae with KaTeX, retrieval strategies, FAQ.

7.9 Frontend - step components

steps/Step1Upload.tsx	Upload + chunker config. Calls POST /ingest, stores corpus_hash in AppContext, advances to Step 2.
steps/Step2Configure.tsx	Strategy multi-select + per-strategy param form + compression toggle + question input. Calls POST /benchmark, stores run_ids, advances to Step 3.
steps/Step3Results.tsx	Polls every run_id in parallel, populates radar / latency / table / score-card / winner widgets as each run completes. Persists history to localStorage.
steps/Step4Chat.tsx	Live chat. Posts to /chat, displays answer with metadata (strategy used, retrieved chunks). Counter and upgrade prompt.

7.10 Frontend - reusable components

Nav.tsx	Global nav with theme toggle, audio toggle, GitHub link, Docs link, Settings (opens BYOKDrawer).
StepIndicator.tsx	Numbered step pills with click-to-go-back.
ParamForm.tsx	Dynamic form built from a param_schema array. Renders sliders for int/float, dropdowns for enum, tooltips from description.
BYOKDrawer.tsx	Slide-in drawer for BYOK key + model. Stores in localStorage. Dispatches a DOM event so AppContext picks up the change without prop drilling.
TierModal.tsx	First-visit modal explaining all three tiers. Stored dismissal in localStorage.
TierSelectionModal.tsx	Landing-page modal for tier selection / sign-up flows.
ErrorBoundary.tsx	React error boundary that wraps each step so a render crash in one step does not unmount the whole app.

ThemeProvider.tsx	Dark / light / system theme management. Persists to localStorage.
AudioManager.tsx	Ambient audio playback + click SFX. Persisted volume / off state in localStorage.
ToastDisplay.tsx	Toast notification renderer. Reads queue from UIContext.
Formula.tsx	KaTeX wrapper for inline and block formulae in the docs page.
SnowflakeBackground.tsx / ParticleBackground.tsx / RAGFlowBackground.tsx	Three decorative canvas backgrounds used on the landing, particles, and docs / chat pages respectively.

7.11 Frontend - state and API

context/AppContext.tsx	useReducer-based store for all per-session benchmark state: corpus_hash, chunk count, selected strategies and params, run_ids, run history (also mirrored to localStorage).
context/UIContext.tsx	Toast queue, drawer open/close, theme accessor, dev mode flag.
lib/api.ts	Typed fetch client. Every backend endpoint typed end-to-end. Auto-attaches X-Dev-Token (from sessionStorage) and X-Fingerprint (from a stable browser hash) headers.
lib/utils.ts	Class-name join and minor formatting helpers.

7.12 Tests, deploy, and supporting files

tests/test_retrieval.py	Unit tests for the four retrieval strategies with mocked LLM.
tests/test_retrieval_strategies.py	Integration-style tests for retrieval that exercise the registry and param filter logic.
tests/test_chunkers.py	Tests for fixed_size, semantic, hierarchical chunkers.
tests/test_ingest.py	Tests for the ingest router including duplicate detection and size limit enforcement.

tests/test_eval.py	Tests for the eval pipeline. Mocks <code>_run_ragas</code> at module level to avoid needing a real OpenAI key.
tests/test_compression.py	Tests for ContextualCompressor.
tests/test_auth.py	Tests for the dev token validation flow.
tests/test_routers.py	Tests for router-level behaviour (rate limits, 404 on missing corpus, 422 on bad input).
tests/parallel_probe.py	Manual stress probe for parallel run handling. Not part of pytest discovery.
scripts/smoke_test.py	End-to-end smoke test against a running local backend and real database. No mocks.
scripts/build_reference_pdf.py	This script. Builds docs/RAGScope_Reference.pdf.
Dockerfile	Railway production image. python:3.11.9-slim base. Installs libpq-dev + gcc for psycopg2-binary. Runs uvicorn on port 8000.
railway.toml	Railway build / deploy / health-check configuration. Pins build to the Dockerfile and binds startCommand to the Railway-injected \$PORT.
docker-compose.yml	Local-dev Postgres + pgvector container on port 5433. Persists data in a named volume.
requirements.txt	Pinned Python dependencies. See Section 8.
frontend/package.json	Pinned Node dependencies and npm scripts.
.env.example	Template for the .env file. Lists every variable the backend reads.
CLAUDE.md	Internal project rules and architectural decisions. The authoritative source for code-style and design constraints.
README.md	Public-facing GitHub README. Quickstart, tier table, stack, deployment, extension pattern.
devlog.md	Append-only narrative log of the sessions that built the project. Useful background for interview prep.

8. Library reference

8.1 Backend - production dependencies

Package	Pinned version	Why it is needed
fastapi	0.115.12	Async web framework. Native pydantic validation, BackgroundTasks, auto OpenAPI docs at /docs.
uvicorn[standard]	0.34.2	ASGI server with uvloop and httptools. Production WSGI replacement for FastAPI.
asyncpg	0.30.0	Async Postgres driver. Used by the request path; binds to the FastAPI main event loop.
psycopg2-binary	2.9.12	Synchronous Postgres driver. Used by the background-task path because it never touches asyncio (see Section 5.4).
pgvector	0.4.0	Postgres vector type bindings for asyncpg and psycopg2. Used for the embedding column on corpus_chunks.
rank-bm25	0.2.2	Pure-Python BM25Okapi implementation. Used by hybrid retrieval. Chosen over Elasticsearch so the project has no extra infra dependency.
openai	1.78.1	Official OpenAI SDK. Currently used indirectly by RAGAS; RAGScope's own provider uses httpx directly.
anthropic	0.52.0	Official Anthropic SDK. Same indirect-via-RAGAS role; BYOK provider uses httpx directly.
httpx	0.28.1	Async + sync HTTP client. The only HTTP library in the backend (no requests, no aiohttp).
ragas	0.1.21	RAG evaluation framework. Pinned to 0.1.21 because the API broke in later versions and the per-metric isolation pattern is keyed to this version's behaviour.
langsmith	0.1.147	Trace export to LangSmith. Used without langchain-core to avoid the langchain monorepo dep.
pydantic-settings	2.9.1	Env-var aware settings class. Powers backend/core/config.py.

python-dotenv	1.1.0	.env file loading in dev. Production reads real env vars; this is a dev convenience.
python-multipart	0.0.27	Required by FastAPI for multipart/form-data parsing on POST /ingest.
pypdf	5.1.0	PDF text extraction. Pure-Python, no system libraries.
datasets	4.8.5	HuggingFace Datasets. Required by RAGAS to wrap the question/answer/contexts triple.
pytest	8.3.5	Test runner.
pytest-asyncio	0.26.0	Adds @pytest.mark.asyncio so async tests run on a managed event loop.

8.2 Frontend - production dependencies

Package	Pinned version	Why it is needed
next	^14.2.29	React framework. App Router, static pre-rendering, route rewrites to the backend, image optimisation.
react / react-dom	^18	The UI library.
framer-motion	^11	Animations and AnimatePresence for step transitions, modal slide-ins, and the winner-badge glow.
recharts	^2	Radar chart, bar chart, and the comparison table tooltip rendering. SVG-based, no canvas.
lucide-react	^0.400.0	Icon set. Tree-shakeable, paired with Tailwind.
katex / @types/katex	^0.16.45 / ^0.16.8	Math typesetting for the metric formulae on the docs page.

8.3 Frontend - dev dependencies

Package	Pinned version	Why it is needed
typescript	^5	TypeScript compiler.
@types/*	^18-^20	Type definitions for Node and React.
eslint	^8	Lint runner.

eslint-config-next	14.2.21	Next.js's official eslint rules.
tailwindcss	^3	Utility-first CSS.
postcss / autoprefixer	^8 / ^10	CSS pipeline used by Tailwind.

9. How the files connect

9.1 Backend import graph

```

backend.main
+-- backend.chunkers.registry      -> imports each chunker module
|   +-- backend.chunkers.fixed_size  (@register)
|   +-- backend.chunkers.semantic    (@register)
|   +-- backend.chunkers.hierarchical (@register)
+-- backend.retrieval.registry     -> imports each retriever module
|   +-- backend.retrieval.naive      (@register)
|   +-- backend.retrieval.hyde       (@register)
|   +-- backend.retrieval.multiquery (@register)
|   +-- backend.retrieval.hybrid     (@register)
|   +-- backend.retrieval.contextual_compression (NOT registered)
+-- backend.retrieval.contextual_compression
+-- backend.core.database          -> get_pool, create_tables, close_pool
+-- backend.routers.ingest         -> POST /ingest
+-- backend.routers.benchmark      -> POST /benchmark
+-- backend.routers.results        -> GET /results/{run_id}
+-- backend.routers.chat           -> POST /chat

backend.eval.ragas_runner          (imported by routers.benchmark)
+-- backend.core.config            -> settings (OPENAI_API_KEY)
+-- backend.core.database          -> make_sync_connection, get_pool
+-- backend.llm.openai_provider    -> OpenAIProvider
+-- backend.retrieval.registry     -> retrieval registry
+-- backend.retrieval.contextual_compression

```

9.2 Why the registries exist

Every retriever, chunker, ingestor, and LLM provider extends a base class and uses the `@register` decorator. The decorator inserts the class into a module-level dict keyed by `cls.name`. The `registry.py` file in each package *imports* all concrete modules so the decorators run at startup. After that, no other code needs to know the list of strategies - everything reads from the populated dict.

This is what makes the API `/strategies` response and the entire frontend dynamic. Adding a fifth retrieval strategy is a one-file change (see Section 12).

9.3 The benchmark call chain

```

HTTP POST /benchmark -> routers/benchmark.create_benchmark()
  validate corpus    -> core/database.corpus_exists()
  validate strategies -> retrieval/registry.registry
  check rate limit   -> core/database.get_run_count()
  for strategy in N:
    insert row        -> pool.fetchrow(INSERT)
    schedule task     -> BackgroundTasks.add_task(eval.run_evaluation, ...)
  increment counter   -> core/database.increment_run_count(delta=N)
  return [run_ids]    -> HTTP 202

```

```

(later, on the worker thread, for each scheduled task:)
eval.run_evaluation() -> creates loop, wraps in Task, drives:
  eval._run_evaluation_async()
  make_sync_connection()      (psycpg2)
  UPDATE benchmark_runs SET status='running'

```

```

SELECT ... FROM corpus_chunks                                (load corpus)
retriever = registry[strategy](corpus, **params)
results = await retriever.retrieve(question, top_k)
if compression_enabled:
    results = await ContextualCompressor.compress(results, question)
answer = _generate_answer(question, contexts, OpenAIProvider())
scores = _run_ragas(question, answer, contexts) (per-metric)
UPDATE benchmark_runs SET status='completed', scores=..., chunks=...

```

9.4 The frontend call chain

```

User opens /app
  AppContextProvider mounts
  TierModal shown if not previously dismissed
  fetch GET /strategies                                (registries -> form schemas)

User clicks "Proceed" on Step 1 with files + chunker config
  POST /ingest multipart                                (file_data, chunker)
  -> response: { corpus_hash, chunk_count }
  AppContext: corpusHash = ...

User clicks "Run" on Step 2 with N strategies + question
  POST /benchmark JSON                                (corpus_hash, strategies)
  -> response: { run_ids: [...] }
  AppContext: runIds = ...
  Step 3 mounted

Step 3 polls each run_id in parallel
  for run_id in runIds:
    setInterval -> GET /results/{run_id} until status terminal
  As each completes, push into runHistory, update charts

User types in Step 4 chat
  POST /chat JSON                                (corpus_hash, q, strategy)
  -> response: { answer, retrieved_chunks, strategy_used }
  decrement local chat counter, render message

```

9.5 Where state lives

- **Server-side, persistent.** Postgres: corpus_chunks (with embeddings), benchmark_runs (with results), rate_limit_counters (per day).
- **Server-side, in-memory.** Nothing. The backend is stateless across requests; every piece of context is reloaded from Postgres.
- **Client-side, persistent.** localStorage: theme, audio settings, BYOK key + model, run history, tier modal dismissal.
- **Client-side, session-only.** sessionStorage: dev token (set from URL param on first visit, cleared on tab close).
- **Client-side, in-memory.** AppContext (corpus_hash, run_ids, currently-displayed runs), UIContext (toast queue, drawer state).

10. Troubleshooting playbook

This is the section to read when something breaks. Each entry lists a symptom, the most likely cause given the free-tier topology, and the fix. References to provider behaviour are taken from the official Railway, Supabase, Vercel, and OpenAI documentation.

10.1 Supabase - free-tier database paused

Symptom. Every backend request hangs for ~30 seconds and then fails. Railway logs show `asyncpg.exceptions.PostgresConnectionError` or `OSError: [Errno 111] Connection refused`.

Cause. Supabase auto-pauses a free-tier project that has had no activity for 7 days. The official Supabase docs (*Database -> Pausing inactive projects*) state the database is taken offline; the pooler accepts connections but the downstream Postgres is unreachable.

Fix.

- Open the Supabase dashboard for the project.
- Click *Restore project* (free-tier projects restore in 1-2 minutes).
- Hit `/health` on the backend to warm the pool.
- **Prevention.** A small scheduled GET to `/health` every few days keeps the pool warm enough that Supabase counts the project as active. Upgrading to the paid tier removes the auto-pause entirely.

10.2 Supabase - 'too many clients already' or connection limits

Symptom. Intermittent backend failures during heavy benchmarking. Errors mention *connection limit exceeded* or *FATAL: sorry, too many clients already*.

Cause. Supabase free tier caps direct connections at 60 (see Supabase docs *Pricing -> Database*). The `asyncpg` default pool is 10 per process; a few cold-boots or stuck background tasks can push you over.

Fix.

- Use the **transaction pooler** URL (port 6543), not the direct connection (port 5432). The pooler multiplexes many client connections over a smaller pool.
- Verify `SUPABASE_URL` in Railway uses port 6543.
- Reduce `make_task_pool()` `max_size` (already 3) if concurrency is exceptional.

10.3 Railway - service spun down on free tier

Symptom. First request after a quiet period takes 15-30 seconds. Subsequent requests are fast.

Cause. Railway's free 'Hobby' allowance allows the service to be evicted from a warm slot when idle. Cold boot needs to pull the image, start uvicorn, and run `create_tables()`.

Fix.

- The 300-second healthcheck timeout in `railway.toml` already accommodates the cold boot.
- If a 5xx is returned during a smoke test, simply retry after 30 seconds.

- **Prevention.** A scheduled /health ping every 5-10 minutes from an external monitor (UptimeRobot, BetterStack) keeps the service warm. Upgrading to the paid Hobby plan removes the eviction behaviour.

10.4 Railway - deploy fails with 'no main.py' or import error

Symptom. Railway deploy logs show `ModuleNotFoundError: No module named 'backend'` or a similar import failure at uvicorn startup.

Cause. Railway is building from the wrong directory or the Dockerfile WORKDIR is wrong.

Fix.

- Confirm the project root in Railway settings is the repo root (not backend/).
- Confirm the Dockerfile sets WORKDIR /app and uses COPY . . so the backend package lands at /app/backend/.
- Run docker build locally and confirm docker run serves /health before redeploying.

10.5 Vercel - frontend build fails on type error

Symptom. Vercel build output shows *Type error* with a path inside frontend/.

Cause. npm run build runs strict TypeScript checks that npm run dev does not. A change that passes locally with dev can break the build.

Fix.

- Run `cd frontend && npm run build` locally before every push. CLAUDE.md mandates this.
- Fix the type error at its source. Do not add `// @ts-ignore` or relax tsconfig.

10.6 Vercel - frontend cannot reach backend

Symptom. The deployed app loads but every API call fails with CORS or network error.

Cause. Two possibilities. (a) NEXT_PUBLIC_API_URL is unset in Vercel settings, so the rewrite in next.config.js falls back to `http://localhost:8000`. (b) The Vercel origin is not in the backend CORS allow-list.

Fix.

- Set NEXT_PUBLIC_API_URL=https://<your-railway-url> in Vercel project settings and redeploy.
- Confirm backend/main.py CORS allow_origins contains your Vercel origin (default includes `https://ragscope.vercel.app` and `http://localhost:3000`).

10.7 OpenAI - invalid_api_key or 401

Symptom. Backend logs show `openai.AuthenticationError` or RAGAS scores return NaN for every run.

Cause. Per the OpenAI docs (*API reference -> Authentication*), 401 means the key is missing, malformed, or revoked.

Fix.

- Confirm OPENAI_API_KEY is set in Railway service variables (not just in local .env).
- Confirm the key starts with `sk-...` and has not been revoked at `platform.openai.com`.

- Confirm the project tied to the key has model access for gpt-4o-mini and text-embedding-3-small.

10.8 OpenAI - 429 rate limit

Symptom. Some benchmark runs fail with status='failed' and error_message contains *Rate limit reached*.

Cause. OpenAI per-key TPM/RPM limit hit (see official *Rate limits* page). The free tier has the strictest ceiling.

Fix.

- Wait 60 seconds and retry the run.
- Reduce per-strategy num_variants on multi-query or top_k if many concurrent runs are in flight.
- Switch to a paid OpenAI tier or BYOK with a different key for the heavy run.

10.9 RAGAS - all metrics return NaN

Symptom. Status='completed' but every metric column is null. The frontend shows '-' for every score.

Cause. Inside _run_ragas, every per-metric evaluate() call raised. Common reasons: missing OPENAI_API_KEY (RAGAS reads it via langchain), the dataset is malformed (empty contexts list), or an upstream API outage.

Fix.

- Check Railway logs for the _run_ragas: metric X raised Y line printed by the logger. The traceback identifies the cause.
- If the cause is auth, see Section 10.7.
- If the cause is empty contexts, check the retriever returned results (chunk count > 0) and the corpus has chunks.

10.10 RAGAS - 'Timeout should be used inside a task'

Symptom. Background task crashes with RuntimeError("Timeout should be used inside a task").

Cause. The legacy Python 3.14 bug described in Section 5.4. *Should* not occur on Railway (Python 3.11.9 pinned in the Dockerfile).

Fix.

- Confirm the running container is built from the pinned Dockerfile and the base image is python:3.11.9-slim.
- If the trace is inside the post-commit cleanup, confirm completed_committed is set in _run_evaluation_async - the outer except checks it and swallows post-commit noise.

10.11 LangSmith - traces missing

Symptom. LangSmith dashboard shows zero traces for the ragscope project.

Cause. Either LANGCHAIN_TRACING_V2 is false in Railway, or LANGCHAIN_API_KEY is missing, or the project name in LANGCHAIN_PROJECT does not match the dashboard.

Fix. Set all three in Railway variables, redeploy.

10.12 Local dev - 'No module named pytest' or 'No module named reportlab'

Cause. The system python is being used instead of the project virtualenv at `.venv/`.

Fix.

- Activate the venv: `source .venv/bin/activate`.
- Or invoke directly: `.venv/bin/python -m pytest / .venv/bin/python scripts/build_reference_pdf.py`.

10.13 Local dev - 'connection refused' on port 5433

Cause. The Docker Postgres container is not running.

Fix.

- `docker-compose up -d` from the repo root.
- `docker ps` to confirm the postgres container has status *healthy*.
- Confirm `SUPABASE_URL` in `.env` points to `postgresql://ragscope:ragscope@localhost:5433/ragscope` for local dev.

10.14 Local dev - 'pgvector extension not found'

Cause. The Postgres image is plain Postgres rather than `pgvector/pgvector:pg16`.

Fix.

- Confirm `docker-compose.yml` uses image `pgvector/pgvector:pg16`.
- `docker-compose down -v && docker-compose up -d` to rebuild against the right image (the `-v` destroys the volume so the new image initialises a fresh DB).

10.15 Frontend - 'CORS policy: No Access-Control-Allow-Origin'

Cause. The frontend origin is not in `backend/main.py`'s CORS allow-list, or a stale service worker is intercepting requests.

Fix.

- Add the new origin to `CORSMiddleware.allow_origins` in `main.py` and redeploy.
- If a stale SW is the cause, open DevTools -> Application -> Service Workers -> Unregister.

10.16 Browser - BYOK key 'leaked' to the backend

Symptom. The user reports that they entered their key in the BYOK drawer but it appears in a request to the RAGScope backend in DevTools Network.

Cause. A regression in `BYOKDrawer.tsx` or `lib/api.ts`. By design, the BYOK key is read in the browser and used to call OpenAI / Anthropic directly. The RAGScope backend never sees it.

Fix.

- Audit BYOKDrawer.tsx: the key must only be written to localStorage and never passed as a fetch body or query parameter to /api/....
- Audit lib/api.ts: no header named X-User-API-Key or similar should be attached.
- If a regression is confirmed, treat it as a security incident: revoke any keys that may have transited the backend, rotate the deploy, and inspect Railway logs (which may have logged request headers in error paths).

11. Operational runbook

11.1 Deploy a backend change

- Run `.venv/bin/python -m pytest` (must show 106+ passing).
- Confirm no debug print statements were re-introduced.
- Commit and push to main. Railway auto-builds from the Dockerfile.
- Watch the Railway deploy logs until the health check passes.
- Hit `/health` from your terminal: expect `{"status":"ok"}`.
- Smoke test: open the live frontend, run a single naive RAG benchmark on a small TXT file.

11.2 Deploy a frontend change

- Run `cd frontend && npm run build`.
- Commit and push. Vercel auto-builds.
- Visit the preview URL Vercel posts on the commit, click through all four steps, confirm no console errors.

11.3 Rotate the dev token

- Generate a new token (any string).
- Update `DEV_TOKEN` in Railway service variables.
- Redeploy (Railway treats env-var changes as triggering redeploys).
- Visit `https://ragscope.vercel.app/app?dev=<new>` to refresh your own access. Old tokens stop working immediately because the backend hashes against the new value.

11.4 Apply a Postgres schema change

- Add the change as a new `conn.execute(...)` in `create_tables()` using `CREATE TABLE IF NOT EXISTS` or `ADD COLUMN IF NOT EXISTS` so the call is idempotent.
- Apply the same SQL via the Supabase SQL editor before redeploying, so the first request after deploy does not race against the in-app migration.

11.5 Investigate a stuck benchmark run

- Query Postgres: `SELECT id, status, created_at, error_message FROM benchmark_runs ORDER BY created_at DESC LIMIT 20;`
- If *pending* for over 5 minutes, the background task was never scheduled (look for an exception in the response path of `/benchmark`).
- If *running* for over 10 minutes, the task crashed between `status=running` and `status=completed` without writing an error. Check Railway logs for the `run_id`.

11.6 Clear the rate-limit table

Run `DELETE FROM rate_limit_counters WHERE date < CURRENT_DATE - INTERVAL '7 days';` via the Supabase SQL editor. Safe to schedule weekly.

12. Interview prep capsule

This is the distilled set of talking points the author keeps ready for any interview that touches RAG, retrieval, FastAPI, async Python, or production LLM evaluation.

12.1 The 60-second pitch

"RAGScope is a public RAG benchmarking harness. You upload a corpus, ask a question, and it runs the same question through four retrieval strategies - naive, HyDE, multi-query, and hybrid BM25 + dense - in parallel, scoring each one with RAGAS on faithfulness, context utilization, answer relevancy, and latency. The frontend is Next.js on Vercel, the backend is FastAPI on Railway, the database is Supabase Postgres with pgvector. Every strategy is auto-discovered through a decorator registry, so adding a fifth one is a single file. The hardest engineering problem was running RAGAS as a background task without deadlocks; the solution was a dedicated event loop per task with a Task wrapper and synchronous psycpg2 for DB access, plus per-metric isolation so one failing metric doesn't kill the whole run."

12.2 The five questions you will be asked

Q. Why are these four retrieval strategies?

Each represents a different *shape* of retrieval: dense baseline (naive), query rewriting (HyDE), query expansion (multi-query), and sparse + dense fusion (hybrid). Picking one of each shape answers the user's real question - which style of retrieval fits my data - rather than ranking minor variations of cosine similarity.

Q. Why pgvector and not Pinecone or Weaviate?

(a) Postgres is already needed for benchmark_runs and rate_limit_counters; pgvector means one database to operate. (b) Supabase free tier covers it. (c) For corpora under ~100k chunks, pgvector is fast enough that the difference is invisible. (d) No client SDK lock-in - any Postgres client works.

Q. Why RAGAS specifically?

It is the de-facto reference-free RAG eval framework. Reference-free matters because RAGScope cannot ask the user for ground-truth answers. Faithfulness, context utilization, and answer relevancy each measure a different failure mode (hallucination, context waste, off-topic), so the three together give a complete picture.

Q. Walk me through the async background task pattern.

FastAPI BackgroundTasks dispatches non-async callables on a worker thread via anyio. The worker thread has no event loop. We create a fresh asyncio loop, wrap the async implementation in a Task via `loop.create_task()` (so `current_task()` is non-None for libraries like `asyncpg` that call `asyncio.timeout()` internally), and drive it with `loop.run_until_complete()`. The task uses synchronous `psycpg2` for DB access because the previous deploy on Python 3.14 hit an `asyncpg` + `asyncio.timeout` interaction that the Task wrapper alone couldn't fix. Each RAGAS metric runs in its own `evaluate()` call wrapped in `try/except BaseException` so one failing metric writes NaN instead of killing the whole run.

Q. How does BYOK keep keys safe?

The key is entered in the BYOK drawer and written to `localStorage` only. On a BYOK request, the *frontend* calls OpenAI or Anthropic directly via `httpx` in the browser - the RAGScope backend never appears in the

request path and so never sees the key. Even server logs of request headers can't capture what was never sent.

12.3 Design decisions worth name-checking

- **Auto-discovery via registry.** Each retriever, chunker, ingestor, provider extends a base class and uses `@register`. The API and UI dynamically reflect everything in the registry, so adding a fifth strategy is a single-file change.
- **Two DB drivers on purpose.** `asyncpg` for the request path (non-blocking, anyio-friendly), `psycopg2` for the background task path (no asyncio entanglement). Different runtimes, different requirements.
- **Per-metric RAGAS isolation.** One `evaluate()` call per metric so a single failing metric does not abort the run. NaN persists as 'NaN':float8 in Postgres and serialises as null over JSON.
- **Fingerprint + IP composite for rate limiting.** Defeats both evasion patterns (clear site data, switch IP).
- **Per-day key with composite primary key.** Rolling reset at midnight UTC without a scheduled cleanup job.
- **Streaming results.** Frontend polls each `run_id` independently and the dashboard updates as each completes - no waiting for the slowest one.
- **Compression as orthogonal axis.** Not a fifth strategy. Toggle independent of strategy choice. Combinable with any of the four.

12.4 Things to bring up unprompted

- **Cold-boot tolerance.** 300s healthcheck on Railway, IF NOT EXISTS on every DDL so `create_tables` is idempotent across cold boots, no in-memory state on the backend at all.
- **Security posture.** CORS allow-list (no wildcard), `hmac.compare_digest` for dev token, BYOK never touches backend, fingerprint hashed before storage, no plaintext PII anywhere.
- **Cost discipline.** One LLM call per chunk in compression, one in HyDE, one for multi-query rewordings; embeddings cached in Postgres so re-uploads short-circuit on the hash.
- **Frontend extension story.** Every parameter form is generated from the backend `param_schema`, so the frontend has zero hardcoded knowledge of any strategy. New strategies render automatically.

12.5 Limitations to acknowledge honestly

- **No streaming generation.** The chat surface gets a single response, not token-by-token streaming. Acceptable for a measurement tool but not for a production assistant.
- **No multi-modal support.** Text only.
- **Single embedding model.** `text-embedding-3-small` only. Adding a second would be straightforward with the registry pattern but is not implemented.
- **RAGAS is itself LLM-judged.** The judge (`gpt-4o-mini`) is an additional source of variance. For research-grade numbers you would use a deterministic eval and multiple judge passes.

- **No ground-truth eval.** `context_precision` is replaced by `context_utilization` because no reference answers are collected. A future version could let users upload a Q&A set for ground-truth scoring.

12.6 If they ask you to extend it on the whiteboard

Add a fifth retrieval strategy. One file. Subclass `BaseRetriever`, set `name`, `display_name`, `description`, `param_schema`. Implement `async retrieve(query, top_k)`. Decorate with `@register`. Add an import line to `backend/retrieval/registry.py`. The API and the frontend pick it up automatically. Write one or two tests in `tests/`.

Add a second LLM provider for BYOK. Same pattern in `backend/llm/`. Implement both `async` and `sync` variants of `complete()` and `embed()`. The frontend BYOK drawer needs an entry in `MODEL_OPTIONS` for the new provider.

Add a new metric. Add it to `metric_defs` in `_run_ragas`. Add a column to `benchmark_runs` via `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` in `create_tables()`. Update the `UPDATE` statement in `_run_evaluation_async` to include the new column. Update the radar / score-card components on the frontend.

End of manual. Re-read Section 5.4 the day before any deploy. Re-read Section 12 the day before any interview.